

Voice Control Color Block Sorting

1. Function Description

After the program starts, voice control is used to issue color block sorting instructions. Upon receiving the instruction, the program controls the robotic arm to grasp the corresponding color block based on the instruction content, and places it at the designated location.

2. Startup and Operation

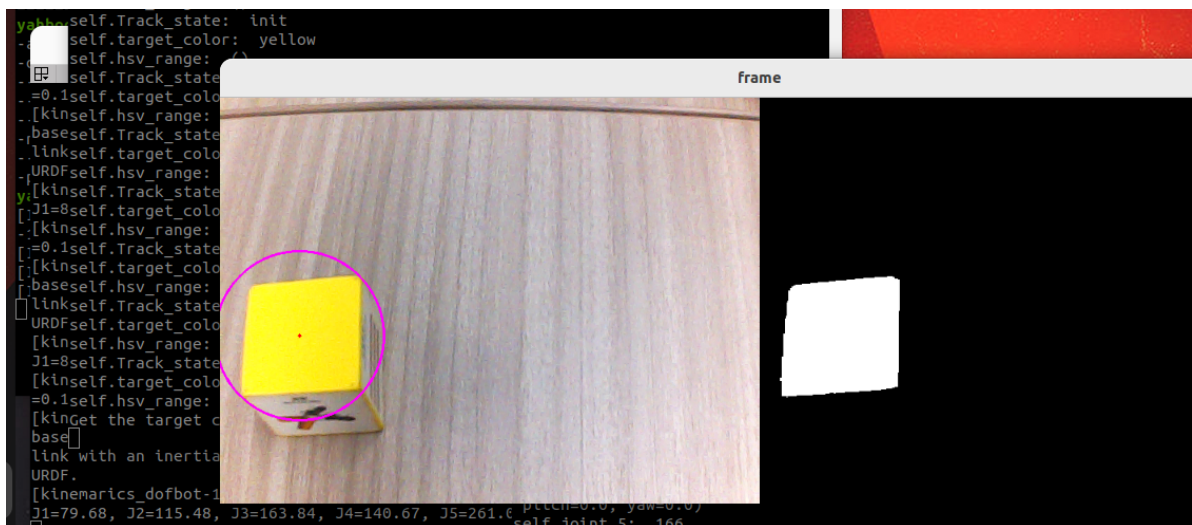
2.1 Startup

Users with Jetson-Nano motherboard version need to enter the docker container and enter the following commands. Orin motherboard users can directly open the terminal and enter the following commands.

```
# Start robotic arm + camera inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start voice recognition program
ros2 launch yahboom_speech speech.launch.py
# Start robotic arm control program
ros2 run dofbot_voice_ctrl grasp_VC
# Start color recognition program
ros2 run dofbot_voice_ctrl color_sorting
```

2.2 Operation Steps

Say "Hi, yahboom" to the voice recognition module, and the speaker will announce "I am here". Then say "Sort the red block"/"Sort the green block"/"Sort the blue block"/"Sort the yellow block" to the voice module. The robotic arm will lower its gripper to grasp the target color block, and the voice module will announce "OK". After grasping the target color block, it will be placed at the designated location. Finally, the robotic arm returns to its initial pose, and the voice announcement module will announce "Placement complete".



3. Core Source Code Analysis

Color Recognition Node color_sorting

Source code path:

```
/home/yahboom/dofbot_ws/src/dofbot_voice_ctrl/dofbot_voice_ctrl/color/color_sorting.py
```

Import necessary libraries

```
import cv2
import os
import numpy as np
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2 as cv
from dofbot_sorting_3d.color_common import *
from dofbot_interface.srv import Kinematics
from dofbot_interface.msg import *
from std_msgs.msg import Bool, Int16, Int8
import time
from rclpy.node import Node
import rclpy
import threading
from Arm_Lib import Arm_Device
print(cv2.__version__)
import os
from dofbot_sorting_3d.compute_joint5 import *
```

Parameter and publisher/subscriber node initialization

```
def __init__(self, name):
    # Initialize the ROS2 node with the given name
    # 使用给定的名称初始化ROS2节点
    super().__init__(name)

    # Create an instance of the robotic arm device class
    # 创建机械臂设备类的实例
    self.Arm = Arm_Device()

    # Define initial joint angles for the robot arm
    # 定义机械臂的初始关节角度
    self.init_joints = [90, 120, 0, 0, 90, 0]

    # Bridge for converting between OpenCV images and ROS2 Image messages
    # 用于在OpenCV图像和ROS2 Image消息之间转换的桥接器
    self.rgb_bridge = CvBridge()

    # Flag indicating whether to publish position information
    # 标志位，表示是否发布位置信息
    self.pub_pos_flag = False

    # Subscription to receive grasp completion status
    # 订阅以接收抓取完成状态
    self.sub_grasp_status = self.create_subscription(
        Bool, "grasp_done", self.get_graspStatusCallback, 100)
```

```

# Publisher for publishing object position information
# 发布物体位置信息的发布者
self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo", 1)

# Publisher for setting joint angle
# 设置关节角度的发布者
self.pub_joint5 = self.create_publisher(Int16, "set_joint5", 10)

# Current target color
# 当前目标颜色
self.target_color = 0

# File paths for HSV color threshold configuration files
# HSV颜色阈值配置文件的路径
self.red_hsv_text = os.path.join(package_pwd, 'red_colorHSV.text')
self.green_hsv_text = os.path.join(package_pwd, 'green_colorHSV.text')
self.blue_hsv_text = os.path.join(package_pwd, 'blue_colorHSV.text')
self.yellow_hsv_text = os.path.join(package_pwd, 'yellow_colorHSV.text')

# Tuple to store HSV range for color filtering
# 存储颜色过滤的HSV范围的元组
self.hsv_range = ()

# Flag for selection state (e.g., selecting ROI)
# 选择状态标志
self.select_flags = False

# Name of the display window
# 显示窗口的名称
self.windows_name = 'frame'

# State machine state for tracking
# 跟踪的状态机状态
self.Track_state = 'init'

# Mouse click position coordinates
# 鼠标点击位置坐标
self.Mouse_XY = (0, 0)

# Image dimensions (columns, rows)
# 图像尺寸（列，行）
self.cols, self.rows = 0, 0

# Initial region of interest (ROI) for color detection
# 颜色检测的初始感兴趣区域（ROI）
self.Roi_init = ()

# Instance of color detection class
# 颜色检测类的实例
self.color = color_detect()

# Currently detected color
# 当前检测到的颜色
self.cur_color = None

# Text color for display
# 显示的文本颜色

```

```
self.text_color = (0, 0, 0)

# Center coordinates of detected object
# 检测到的物体中心坐标
self.cx = 0
self.cy = 0

# Radius of detected circular object
# 检测到的圆形物体的半径
self.circle_r = 0

# Message for publishing joint angle
# 发布关节角度的消息
self.joint5 = Int16()

# Array to store corner points of detected object
# 存储检测到的物体角点的数组
self.corners = np.empty((4, 2), dtype=np.int32)

# Current target color index
# 当前目标颜色索引
self.cur_target_color = 0

# Flag for updating HSV thresholds
# 更新HSV阈值的标志
self.updata_flag = False

# Move arm to initial position on startup
# 启动时将机械臂移动到初始位置
self.Arm.Arm_serial_servo_write6_array(self.init_joints, 2000)

# Default distance parameter
# 默认距离参数
self.dist = 0.13

# Subscription to receive raw camera images
# 订阅以接收原始相机图像
self.subscription = self.create_subscription(
    Image, '/image_raw', self.ImageCallback, 1)

# Subscription to receive voice recognition results
# 订阅以接收语音识别结果
self.sub_voice = self.create_subscription(
    Int8, "voice_result", self.getVoiceResultCallback, 1)

# Publisher for playing audio
# 发布音频发布者
self.pub_playID = self.create_publisher(Int8, "player_id", 1)

# Flag for enabling position publishing
# 启用位置发布的标志
self.pubPos_flag = False

# Print initialization complete message
# 打印初始化完成消息
print("Init done.")
```

Image subscription callback function

```
def ImageCallback(self, color_frame):
    # Convert ROS2 Image message to OpenCV RGB image format
    # 将ROS2 Image消息转换为OpenCV RGB图像格式
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')

    # Convert RGB image to BGR format for OpenCV processing
    # 将RGB图像转换为BGR格式以进行OpenCV处理
    rgb_image = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)

    # Create a copy of the image for drawing results
    # 创建图像副本用于绘制结果
    result_image = np.copy(rgb_image)

    # Capture key press
    # 捕获按键
    key = cv2.waitKey(10) & 0xFF

    # Process the image to detect colors and get binary mask
    # 处理图像以检测颜色并获取二值掩码
    result_frame, binary = self.process(rgb_image)

    # Start a separate thread to display the image
    # 启动独立线程显示图像
    # This prevents blocking the main callback loop
    # 这可以防止阻塞主回调循环
    show_frame = threading.Thread(target=self.img_out, args=(result_frame,
binary,))
    show_frame.start()
    show_frame.join() # Wait for the display thread to finish

    # Check if a valid object (with sufficient size) has been detected
    # 检查是否检测到有效物体（具有足够大小）
    # center coordinates non-zero, radius > 30 pixels
    # 中心坐标非零，半径 > 30像素
    if self.cx != 0 and self.cy != 0 and self.circle_r > 30:
        # Convert center coordinates to integers
        # 将中心坐标转换为整数
        cx = int(self.cx)
        cy = int(self.cy)

        # Calculate normalized coordinates (a, b)
        # 计算归一化坐标(a, b)
        # a: horizontal offset from image center, normalized
        # a: 相对于图像中心的水平偏移，已归一化
        # b: vertical offset from image center, normalized with scaling factor
        # b: 相对于图像中心的垂直偏移，带缩放因子归一化
        (a, b) = (round(((cx - 320) / 4000), 5), round(((480 - cy) / 3000) * 0.8
+ 0.13, 5))

        # Calculate edge vector from detected object corners
        # 从检测到的物体角点计算边缘向量
        vx = self.corners[0][0][0] - self.corners[1][0][0]
        vy = self.corners[0][0][1] - self.corners[1][0][1]

        # Compute the required angle for joint 5 based on edge orientation
```

```

# 根据边缘方向计算关节5所需的角速度
target_joint5 = compute_joint5(vx, vy)
print("target_joint5: ", target_joint5)

# If position publishing is enabled
# 如果位置发布已启用
if self.pubPos_flag == True:
    # Disable further position publishing
    # 禁用位置发布
    self.pubPos_flag = False

    # Create and publish joint command
    # 创建并发布关节控制命令
    joint5 = Int16()
    joint5.data = int(target_joint5)
    self.pub_joint5.publish(joint5)

    # Create and publish object position information
    # 创建并发布物体位置信息
    pos = AprilTagInfo()
    pos.id = self.cur_target_color
    pos.x = float(b)
    pos.y = float(a)
    pos.z = 0.03
    self.pos_info_pub.publish(pos)

# If object is too small (radius < 30) and position publishing is still
enabled
# 如果物体太小（半径 < 30）且位置发布仍处于启用状态
elif self.circle_r < 30 and self.pubPos_flag == True:
    # Reset the system to initial state
    # 将系统重置为初始状态
    self.Reset()

    # Disable further position publishing
    # 禁用位置发布
    self.pubPos_flag = False

    # Send play ID
    # 发送播放ID
    play_id = Int8()
    play_id.data = 1
    self.pub_playID.publish(play_id)

```

ROI image processing function

```

def process(self, rgb_img):
    # Resize the input image to 640x480 resolution
    # 将输入图像调整为640x480分辨率
    rgb_img = cv.resize(rgb_img, (640, 480))

    # Initialize empty binary image list
    # 初始化二值图像列表
    binary = []

    # State machine: Initialization state
    # 状态机：初始化状态

```

```

if self.Track_state == 'init':
    # Create a named window for mouse interaction
    # 创建用于鼠标交互的命名窗口
    cv.namedWindow(self.windows_name, cv.WINDOW_AUTOSIZE)
    # Set mouse callback for selecting ROI
    # 设置鼠标回调以选择感兴趣区域
    cv.setMouseCallback(self.windows_name, self.onMouse, 0)

    # If ROI selection is active, draw guides on the image
    # 如果ROI选择处于活动状态，在图像上绘制引导线
    if self.select_flags == True:
        # Draw a line from start to current mouse position
        # 从起点到当前鼠标位置绘制线条
        cv.line(rgb_img, self.cols, self.rows, (255, 0, 0), 2)
        # Draw a rectangle for the ROI
        # 为ROI绘制矩形
        cv.rectangle(rgb_img, self.cols, self.rows, (0, 255, 0), 2)

        # If ROI is valid (width and height > 0)
        # 如果ROI有效（宽度和高度 > 0）
        if self.Roi_init[0] != self.Roi_init[2] and self.Roi_init[1] !=
self.Roi_init[3]:
            # Extract HSV range from the selected ROI
            # 从选定的ROI中提取HSV范围
            rgb_img, self.hsv_range = self.color.Roi_hsv(rgb_img,
self.Roi_init)
            self.dyn_update = True
        else:
            # Invalid ROI, stay in init state
            # ROI无效，保持在初始化状态
            self.Track_state = 'init'

# State machine: Identify/Tracking state
# 状态机：识别/跟踪状态
elif self.Track_state == "identify":
    # Load HSV range based on selected color
    # 根据选择的颜色加载HSV范围
    if self.target_color == 3:
        self.hsv_range = read_HSV(self.red_hsv_text)
        self.cur_color = "red"
        self.text_color = (255, 0, 0) # Red text (BGR)
    elif self.target_color == 2:
        self.hsv_range = read_HSV(self.green_hsv_text)
        self.cur_color = "green"
        self.text_color = (0, 255, 0) # Green text (BGR)
    elif self.target_color == 1:
        self.hsv_range = read_HSV(self.blue_hsv_text)
        self.cur_color = "blue"
        self.text_color = (0, 0, 255) # Blue text (BGR)
    elif self.target_color == 4:
        self.hsv_range = read_HSV(self.yellow_hsv_text)
        self.cur_color = "yellow"
        self.text_color = (255, 255, 0) # Yellow text (BGR)

# Object detection and tracking (when not in init state)
# 物体检测与跟踪（不在初始化状态时）
if self.Track_state != 'init':
    # Ensure HSV range is loaded before detection

```

```
# 确保在检测前已加载HSV范围
if len(self.hsv_range) != 0:
    # Perform color detection
    # 执行颜色检测
    rgb_img, binary, self.circle, _, self.corners =
self.color.object_follow(rgb_img, self.hsv_range)

    # Extract circle parameters
    # 提取圆形参数
    self.cx = self.circle[0] # Center x
    self.cy = self.circle[1] # Center y
    self.circle_r = self.circle[2] # Radius

    # Update HSV configuration file if flag is set
    # 如果标志已设置，则更新HSV配置文件
    if self.cur_target_color == 3 and self.updata_flag == True:
        write_HSV(self.red_hsv_text, self.hsv_range)
    elif self.cur_target_color == 2 and self.updata_flag == True:
        write_HSV(self.green_hsv_text, self.hsv_range)
    elif self.cur_target_color == 1 and self.updata_flag == True:
        write_HSV(self.blue_hsv_text, self.hsv_range)
    elif self.cur_target_color == 4 and self.updata_flag == True:
        write_HSV(self.yellow_hsv_text, self.hsv_range)
    self.updata_flag = False # Reset update flag

# Overlay the current color name on the image
# 在图像上叠加当前颜色名称
rgb_img = cv2.putText(rgb_img, self.cur_color, (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 1, self.text_color, 2)

# Return the processed image and binary mask
# 返回处理后的图像和二值掩码
return rgb_img, binary
```